Experiment 6

Aim :

To create a stored procedure transfer_employee(emp_id, new_dept_id) with validation and error handling, and to implement database triggers for salary validation and audit logging.

Description :

A stored procedure is a precompiled collection of SQL statements that can be executed as a single unit. In this experiment, a stored procedure named transfer_employee is used to transfer an employee from one department to another. The procedure accepts an employee ID and a new department ID as parameters.

The procedure validates whether the employee exists and whether the destination department exists before performing the transfer. Transaction control and error handling are used so that the operation is rolled back if an error occurs.

Triggers are special database programs that execute automatically when specified events occur on a table. A salary validation trigger is implemented to prevent invalid or negative salary values. An audit trigger records salary changes in a separate audit table so that changes can be tracked.

The experiment also tests valid transfers, invalid employee IDs, invalid department IDs, invalid salary values, and salary update auditing. The following SQL uses MySQL 8.0 or later.

Source Code :

```
-- Create department table

CREATE TABLE department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL
);

-- Create employee table

CREATE TABLE employee (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(100) NOT NULL,
    department_id INT,
    salary DECIMAL(10,2),
    FOREIGN KEY (department_id) REFERENCES department(department_id)
);

-- Insert sample departments

INSERT INTO department VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');
-- Insert sample employees

INSERT INTO employee VALUES
```

```
(101, 'Amit', 1, 90000),
(102, 'Neha', 1, 65000),
(103, 'Rahul', 2, 60000),
(104, 'Priya', 3, 80000);

-- Create audit table

CREATE TABLE salary_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Stored procedure for employee transfer

DELIMITER //

CREATE PROCEDURE transfer_employee(
    IN emp_id INT,
    IN new_dept_id INT
)
BEGIN
    DECLARE employee_count INT DEFAULT 0;
    DECLARE department_count INT DEFAULT 0;

    -- Error handler
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Transfer failed. Transaction rolled back.' AS message;
    END;

    START TRANSACTION;

    -- Validate employee
    SELECT COUNT(*) INTO employee_count
    FROM employee
    WHERE employee_id = emp_id;

    IF employee_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee does not exist';
    END IF;
    -- Validate department
    SELECT COUNT(*) INTO department_count
    FROM department
    WHERE department_id = new_dept_id;
```

```
    IF department_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Department does not exist';
    END IF;

    -- Transfer employee
    UPDATE employee
    SET department_id = new_dept_id
    WHERE employee_id = emp_id;

    COMMIT;

    SELECT 'Employee transferred successfully.' AS message;
END //

DELIMITER ;

-- Test valid employee transfer

CALL transfer_employee(102, 2);


SELECT * FROM employee
WHERE employee_id = 102;

-- Test invalid employee ID

CALL transfer_employee(999, 2);

-- Test invalid department ID

CALL transfer_employee(102, 99);

-- Trigger for salary validation

DELIMITER //

CREATE TRIGGER validate_salary_before_insert
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than zero';
    END IF;
END //

DELIMITER ;

-- Trigger for salary validation during update
```

```
DELIMITER //

CREATE TRIGGER validate_salary_before_update
BEFORE UPDATE ON employee
FOR EACH ROW
BEGIN
   IF NEW.salary <= 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Salary must be greater than zero';
   END IF;
END //

DELIMITER ;

-- Trigger for salary audit logging

DELIMITER //

CREATE TRIGGER salary_audit_after_update
AFTER UPDATE ON employee
FOR EACH ROW
BEGIN
   IF OLD.salary <> NEW.salary THEN
      INSERT INTO salary_audit
      (employee_id, old_salary, new_salary)
      VALUES
      (OLD.employee_id, OLD.salary, NEW.salary);
   END IF;
END //

DELIMITER ;

-- Test salary validation

UPDATE employee
SET salary = -5000
WHERE employee_id = 101;

-- Test valid salary update and audit logging

UPDATE employee
SET salary = 95000
WHERE employee_id = 101;

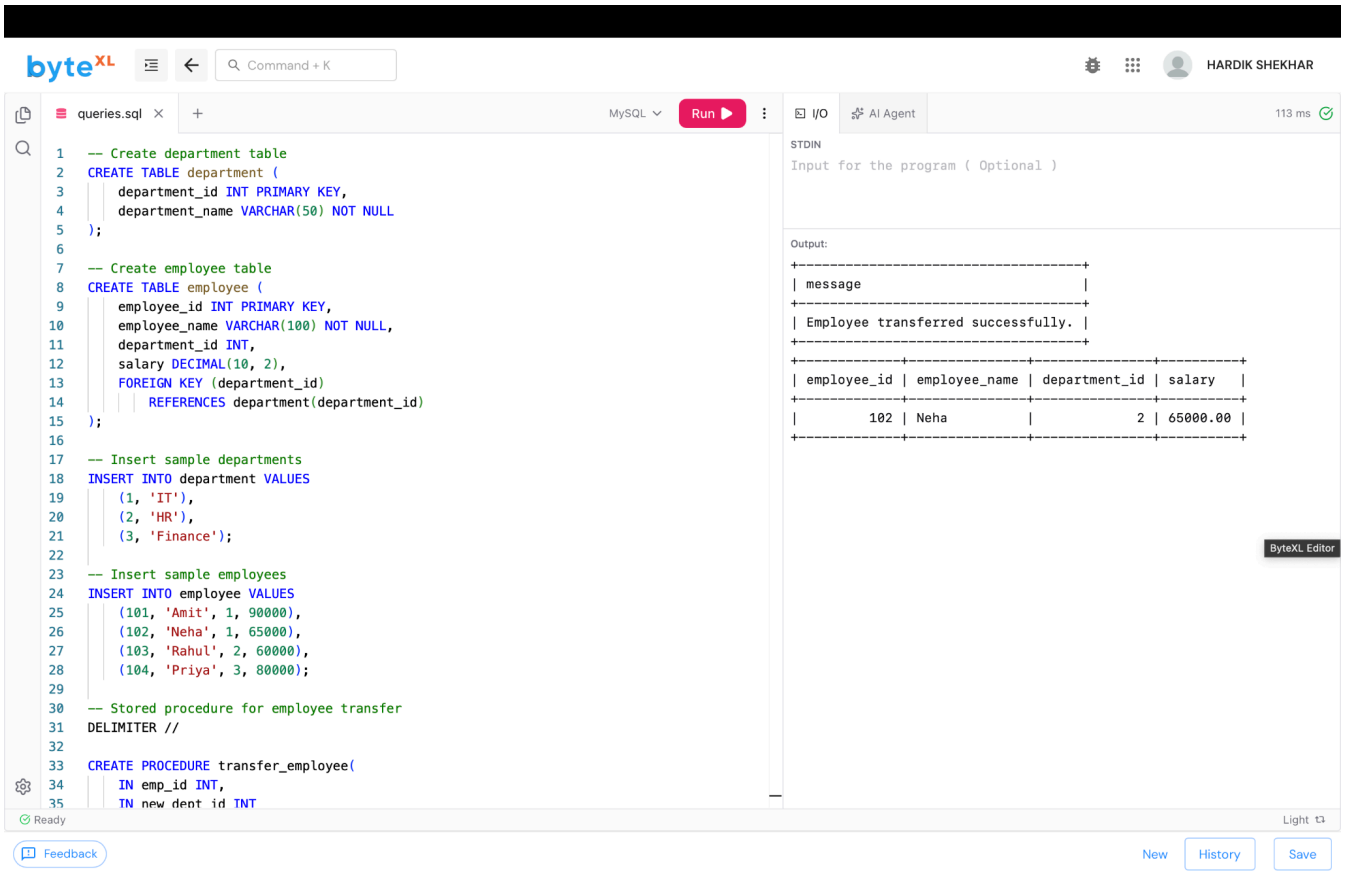
-- Display audit records

SELECT * FROM salary_audit;
```

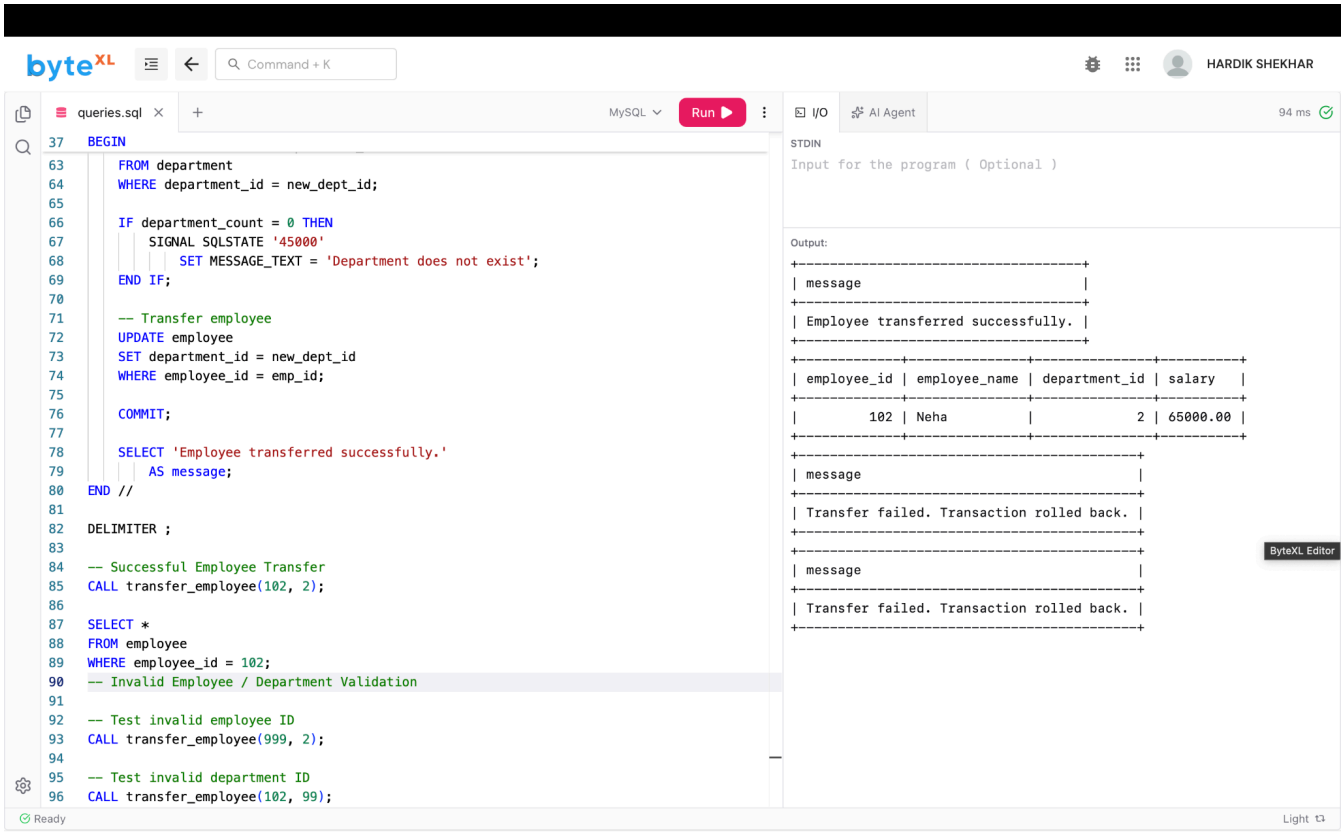
Output :

Execute the SQL statements and capture screenshots showing the successful employee transfer, validation/error messages, salary validation, and salary audit records.

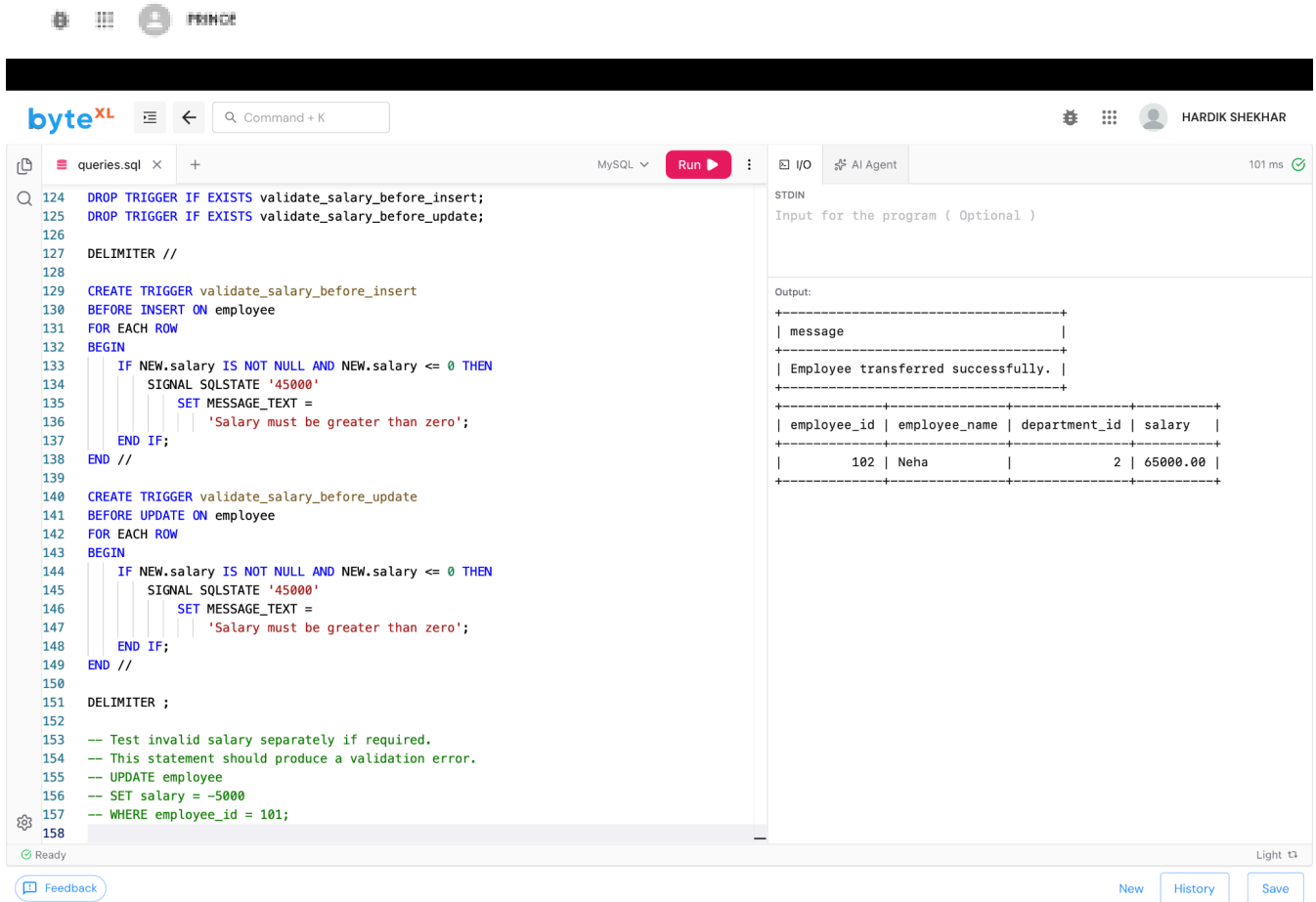
Screenshot 1: Successful Employee Transfer



Screenshot 2: Invalid Employee / Department Validation



Screenshot 3: Salary Validation Trigger



Screenshot 4: Salary Audit Log

